# Multi-Bit Ripple-Carry Adders Workshop

Student Lab Handout

## Session overview

**Duration:** 1h15m

**Format:** Paired coding .

### Session Objectives

- Construct and test a multi-bit ripple-carry adder in Qiskit.
- Start from the reversible 1-bit full adder
- Extend the design to four bits and analyze performance, resource usage, and correctness.
- Advanced exercises involving larger adders and IBM Quantum hardware execution

# Background

A classical full adder adds three input bits $A, B, \text{Cin}$ and produces a sum and carry:

$$\text{SUM} = A \oplus B \oplus \text{Cin}, \qquad \text{Cout} = AB + A\text{Cin} + B\text{Cin}.$$

Quantum circuits must be reversible, so instead of XOR and AND we use:

- XOR $\rightarrow$ CNOT
- AND $\rightarrow$ Toffoli (CCNOT)

## Numerical Example of Full-Adder Operation

To illustrate the operation of a classical full adder, consider the three input bits

$$A = 1, \qquad B = 1, \qquad \text{Cin} = 0.$$

A full adder computes two outputs:

$$\text{SUM} = A \oplus B \oplus \text{Cin}, \qquad \text{Cout} = AB + A\text{Cin} + B\text{Cin}.$$

### Integer Interpretation

We may interpret the three input bits as simple integers and add them:

$$1 + 1 + 0 = 2.$$

The result, 2, will be converted into its binary representation to relate it to the full-adder outputs.

### Binary Representation

The decimal number 2 corresponds to the binary number

$$2_{10} = 10_2 = 1 \times 2^1 + 0 \times 2^0.$$

This two-bit binary value separates naturally into

$$\text{Cout} = 1, \qquad \text{SUM} = 0,$$

since, in binary, the leftmost bit represents the carry-out and the rightmost bit represents the least significant bit (the sum).

### Logical Verification

To confirm this result using Boolean logic, we compute the SUM and CARRY explicitly.

**SUM computation:**

$$A \oplus B = 1 \oplus 1 = 0, \qquad 0 \oplus \text{Cin} = 0 \oplus 0 = 0.$$

Thus,

$$\text{SUM} = 0.$$

**CARRY computation:** We evaluate each product term:

$$AB = 1 \cdot 1 = 1, \qquad A\text{Cin} = 1 \cdot 0 = 0, \qquad B\text{Cin} = 1 \cdot 0 = 0.$$

Therefore,

$$\text{Cout} = AB + A\text{Cin} + B\text{Cin} = 1 + 0 + 0 = 1.$$

**Final Result**

Combining these results, we obtain

$$(\text{Cout}, \text{SUM}) = (1, 0),$$

which is exactly the binary representation of the integer sum:

$$1 + 1 + 0 = 2 = 10_2 = (\text{Cout} = 1,\ \text{SUM} = 0).$$

This example illustrates how a full adder decomposes binary addition into separate sum and carry bits.

# Setup

Run the following first:

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

sim = AerSimulator()
print("Qiskit 2.x Ripple-Carry Adder Lab Ready")
```

# 1 The 1-Bit Full Adder

## Mathematical Logic

A reversible full adder takes three inputs:

$$A, \quad B, \quad \text{Cin},$$

and produces two outputs:

$$\text{SUM}, \qquad \text{Cout}.$$

The SUM is:

$$\text{SUM} = A \oplus B \oplus \text{Cin}.$$

The carry-out is:

$$\text{Cout} = AB + A\text{Cin} + B\text{Cin}.$$

## 1.1 Reversible Qiskit Implementation

```
def full_adder(qc, a, b, cin, sum_qubit, cout_qubit):

    # SUM logic
    qc.cx(a, sum_qubit)
    qc.cx(b, sum_qubit)
    qc.cx(cin, sum_qubit)

    # CARRY logic
    qc.ccx(a, b, cout_qubit)
    qc.ccx(a, cin, cout_qubit)
    qc.ccx(b, cin, cout_qubit)
```

## 1.2 Circuit Setup

```
qc = QuantumCircuit(5, 2)
# 0: A
# 1: B
# 2: Cin
# 3: Sum
# 4: Cout
```

## 1.3 Input Preparation

```
# Example Input: A = 1, B = 1, Cin = 0
qc.x(0)   # A
qc.x(1)   # B
# Cin remains 0
```

## 1.4 SUM Logic

```
qc.cx(0, 3)
qc.cx(1, 3)
qc.cx(2, 3)
```

## 1.5 Carry Logic

```
qc.ccx(0,1,4)
qc.ccx(0,2,4)
qc.ccx(1,2,4)
```

## 1.6 Measurement

```
qc.measure(3,0)
qc.measure(4,1)
```

## 1.7 Simulation

Circuit diagram showing SUM computed from three CNOT gates, and CARRY computed from three Toffoli gates, with measurements on SUM and CARRY lines.

```
simulator = AerSimulator()
compiled = transpile(qc, simulator)
result = simulator.run(compiled, shots=1024).result()
print(result.get_counts())
```

Output example:

$$\{\text{"10"} : 1024\}$$

## 1.8 Truth Table Generator

```
def run_full_adder(A, B, Cin):

    qc = QuantumCircuit(5, 2)

    if A: qc.x(0)
    if B: qc.x(1)
```

```
    if Cin: qc.x(2)

    qc.cx(0,3)
    qc.cx(1,3)
    qc.cx(2,3)

    qc.ccx(0,1,4)
    qc.ccx(0,2,4)
    qc.ccx(1,2,4)

    qc.measure(3,0)
    qc.measure(4,1)

    sim = AerSimulator()
    compiled = transpile(qc, sim)
    result = sim.run(compiled, shots=1).result()

    bits = list(result.get_counts().keys())[0]
    return int(bits[0]), int(bits[1])
```

```
for A in [0,1]:
    for B in [0,1]:
        for Cin in [0,1]:
            print(A, B, Cin, "->", run_full_adder(A,B,Cin))
```

## Full Adder Truth Table

The following table shows the complete truth table for a classical full adder. The inputs are the three bits $A, B,$ Cin, and the outputs are the sum bit SUM and carry-out bit Cout.

| A | B | Cin | SUM | Cout |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

Example: row with A=1, B=1, Cin=0
Binary input: $110 \mapsto$ Decimal input: $1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 4 + 2 = 6$
Binary output: $10 \mapsto$ Decimal output: $1 \times 2^1 + 0 \times 2^0 = 2$
This matches: $1 + 1 + 0 = 2$
where:
Sum $\rightarrow$ least significant bit $\rightarrow 0$
Cout $\rightarrow$ most significant bit $\rightarrow 1$

| A | B | Cin | SUM | Cout | Binary Output | Decimal Output |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | | | | |
| 0 | 0 | 1 | | | | |
| 0 | 1 | 0 | | | | |
| 0 | 1 | 1 | | | | |
| 1 | 0 | 0 | | | | |
| 1 | 0 | 1 | | | | |
| 1 | 1 | 0 | | | | |
| 1 | 1 | 1 | | | | |

Table 1: Full Adder Truth Table with Binary and Decimal Outputs

### Exercises

Using the code above, calculate all possible input combinations for a 1-bit Full Adder to verify the logic. For each row, calculate:

- The binary Sum and Carry-Out.
- The resulting Binary Result string.
- The Decimal Equivalent of that binary string.

## 2 Multi-Bit Ripple-Carry Quantum Adder Using Qiskit

We extend the reversible 1-bit full adder into a multi-bit *ripple-carry adder*. This design is used in classical processors and translates naturally into reversible logic, making it ideal for quantum circuit construction in Qiskit. A ripple-carry adder operates by chaining multiple full adders so that the carry-out from each bit becomes the carry-in to the next.

- How a reversible 1-bit full adder functions.
- How to build an $n$-bit ripple-carry adder using this building block.
- How to implement and test the adder in Qiskit 2.x.
- A detailed worked example: $0110_2 + 1001_2 = 1111_2$.

### Reversible 1-Bit Full Adder

The full adder takes three inputs:

$$A, \quad B, \quad \text{Cin},$$

and produces:

$$\text{SUM} = A \oplus B \oplus \text{Cin}, \qquad \text{Cout} = AB + A\,\text{Cin} + B\,\text{Cin}.$$

The reversible Qiskit implementation is:

Listing 1: Reversible 1-bit full adder

```
def full_adder(qc, a, b, cin, sum_qubit, cout_qubit):
    # SUM = A XOR B XOR Cin
    qc.cx(a, sum_qubit)
    qc.cx(b, sum_qubit)
    qc.cx(cin, sum_qubit)

    # CARRY = AB + A*Cin + B*Cin
    qc.ccx(a, b, cout_qubit)
    qc.ccx(a, cin, cout_qubit)
    qc.ccx(b, cin, cout_qubit)
```

This block will be used repeatedly inside the multi-bit adder.

## Ripple-Carry Adder Design

An $n$-bit ripple-carry adder adds two binary numbers:

$$A = (A_{n-1} \dots A_1 A_0), \qquad B = (B_{n-1} \dots B_1 B_0),$$

where bit 0 is the least significant bit (LSB).

The algorithm is:

1. Set the initial carry-in: $\text{Cin}_0 = 0$.
2. For each bit $i$ from 0 to $n-1$:
$$(\text{SUM}_i,\ \text{Cout}_i) = \text{FullAdder}(A_i, B_i, \text{Cin}_i).$$
3. Propagate:
$$\text{Cin}_{i+1} = \text{Cout}_i.$$
4. The final output consists of:
$$\text{SUM}_{n-1} \dots \text{SUM}_0, \qquad \text{Cout}_n.$$

## Qiskit Implementation of the Ripple-Carry Adder

The following function automatically chains $n$ full adders:

Listing 2: Ripple-carry adder function

```
def ripple_carry_adder(qc, A, B, Cin, Sum, Cout, n):
    carry = Cin  # initial carry-in

    for i in range(n):
        next_carry = qc.num_qubits - (i+1)
        full_adder(qc, A[i], B[i], carry, Sum[i], next_carry)
        carry = next_carry

    qc.cx(carry, Cout)   # final carry-out
```

## Example: 4-Bit Addition

We add:

$$A = 0110_2 = 6, \qquad B = 1001_2 = 9,$$

so the correct result is:

$$A + B = 1111_2 = 15.$$

Below is a fully-working 4-bit test:

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

qc = QuantumCircuit(18, 5)

A_bits = [0,1,2,3]
B_bits = [4,5,6,7]
Sum_bits = [8,9,10,11]
Cin = 12
Cout = 17

# Initialize A = 0110
qc.x(A_bits[1])
qc.x(A_bits[2])

# Initialize B = 1001
qc.x(B_bits[0])
qc.x(B_bits[3])

# Build ripple-carry adder
ripple_carry_adder(qc, A_bits, B_bits, Cin, Sum_bits, Cout, n=4)

# Measure
qc.measure(Sum_bits[0], 0)
qc.measure(Sum_bits[1], 1)
qc.measure(Sum_bits[2], 2)
qc.measure(Sum_bits[3], 3)
qc.measure(Cout, 4)

# Run
sim = AerSimulator()
compiled = transpile(qc, sim)
result = sim.run(compiled, shots=1024).result()

print(result.get_counts())
```

Expected output:

$$\text{Cout SUM} = 0\,1111_2 = 15.$$

## Exercises

After implementing the ripple-carry adder, you should complete:

1. **Verification**: Test all pairs of 4-bit numbers and compare to Python's:

   `int(A,2) + int(B,2)`.

2. **Circuit Diagram**: Draw the circuit using:

   ```
   qc.draw("mpl")
   ```

   and annotate the carry flow.

3. **Gate Count and Depth**: Use:

   ```
   qc.count_ops(), qc.depth()
   ```

   and discuss the cost of reversible logic.

4. **Scaling Analysis**: Explain how the number of Toffoli gates grows with bit-width.

5. **Challenge**: Extend your adder to 8 bits or 10 bits.